

Building an Observable Local LLM Platform

After deploying the AI platform architecture with Kubernetes, the final step is bringing the full system together into one integrated local platform.

Kubernetes defines the deployment boundaries. The capstone asks a broader question:

Can the platform behave like a production AI system, even while running locally?

That is the purpose of the Observable Local LLM Platform.

This project brings together the major pieces of the AI Infrastructure Learning Lab into a single architecture: routing, retrieval, model serving, background processing, metrics, benchmarking, Docker Compose, and Kubernetes deployment readiness.

From an AI Data Platform Architect's perspective, this capstone matters because modern AI systems are not just model endpoints. They are platforms made of service boundaries, source-grounded retrieval, measured inference behavior, async execution, operational telemetry, and deployment controls.

The model powers the experience.

The platform makes the experience dependable.

What the Platform Does

The platform accepts a user question through an API gateway.

The gateway orchestrates the request by calling the retrieval service, building a source-grounded prompt, calling the LLM service, recording latency and token metrics, and returning an answer with source citations and reliability fields.

At a high level, the request flow looks like this:

```
User question
-> API gateway
-> RAG service retrieves source context
-> gateway builds grounded prompt
-> LLM service generates answer
-> observability layer records metrics
-> response returns answer, sources, and reliability fields
```

The platform also includes:

- an async worker path for long-running jobs
- Redis for queue coordination
- Postgres for durable data and pgvector-ready storage
- a local observability dashboard
- a benchmark script for the `/ask` endpoint
- Docker Compose for local integration
- Kubernetes manifests for production-style deployment boundaries

The result is a local system that behaves like a small AI platform instead of a single application.

That distinction is important.

A single AI application may be able to answer a question.

An AI platform needs to answer the question, explain where the answer came from, measure how long each dependency took, track token usage, support async jobs, and provide a path toward deployment and scaling.

Why This Architecture Matters

An AI application becomes difficult to operate when too many responsibilities live in one process.

If one service handles routing, retrieval, generation, document storage, metrics, background processing, and deployment concerns, the system becomes hard to scale, debug, and harden.

The capstone separates those responsibilities.

The API gateway owns request validation and orchestration.

The RAG service owns document retrieval.

The LLM service owns model generation and token accounting.

The worker owns asynchronous execution.

Redis owns queue coordination.

Postgres represents durable data and pgvector-ready storage.

The observability layer owns metrics and dashboard visibility.

Kubernetes owns deployment boundaries, routing, scaling, and network policy.

This separation is the core architectural idea.

Each service has a clear responsibility.

Each dependency has a clear purpose.

Each operational signal has a place to land.

That makes the platform easier to reason about.

It also creates a foundation that can evolve. A mock LLM service can later become a real local model runtime or GPU-backed serving layer. A local Postgres instance can later become managed Postgres. A local dashboard can later become a managed observability stack. Docker Compose can support local integration while Kubernetes manifests define the production-style deployment shape.

The capstone is not pretending to be the final production environment.

It is proving the platform shape.

The API Gateway as the Orchestration Layer

The API gateway is the entry point for user-facing requests.

Its job is not to perform every task directly. Its job is to coordinate the platform.

When a user submits a question, the gateway is responsible for:

- validating the request
- calling the RAG service for relevant context
- building a source-grounded prompt
- calling the LLM service for generation
- collecting latency and token metrics
- returning the final answer with sources and reliability fields

This is a strong platform pattern.

The gateway owns orchestration, not every implementation detail.

That separation keeps the system modular. Retrieval logic can improve without rewriting the gateway. Model-serving behavior can change without rewriting document search. Observability fields can expand without collapsing the service boundary.

From an architecture perspective, the gateway is the control point for the user-facing AI workflow.

It defines how the platform turns a question into a measured, source-grounded answer.

The RAG Service as the Retrieval Layer

The RAG service owns document retrieval.

Its role is to find relevant source chunks that can support the answer. This keeps retrieval as a dedicated platform capability instead of burying it inside the model-serving layer.

That matters because retrieval quality is one of the most important determinants of answer quality.

The RAG service is responsible for the context that the model receives. If retrieval is weak, the model may produce an unsupported or low-value answer. If retrieval is strong, the model has a better chance of producing a grounded response.

In the capstone architecture, the RAG service provides the source material used to construct the final prompt.

The response includes source chunks and retrieval similarity scores. That gives the platform a way to inspect whether the answer was based on relevant evidence.

This is a critical difference between a generic chatbot and a source-grounded AI assistant.

The assistant should not only produce text.

It should expose the sources that shaped the answer.

The LLM Service as the Generation Layer

The LLM service owns model generation and token accounting.

That boundary is important because generation has a different operating profile than retrieval, routing, or background work.

Model-serving workloads are sensitive to prompt size, output length, latency, token throughput, model configuration, and runtime behavior. Keeping generation behind its own service allows the platform to measure and eventually scale it independently.

In the capstone, the LLM service receives a grounded prompt and returns generated output along with model-related fields.

The platform captures:

- model name
- model latency

- prompt tokens
- completion tokens
- total tokens

Those fields matter because AI systems need more than answer text.

They need operational visibility into the cost and performance characteristics of generation.

A response that uses too many tokens, takes too long, or depends on the wrong model configuration may still be technically successful, but it may not be acceptable from a platform reliability perspective.

The LLM service boundary gives the architecture a clean place to measure that behavior.

The AI Reliability Loop

One of the most important ideas in the capstone is that the `/ask` endpoint returns more than an answer.

It returns the operational data needed to understand the answer.

The response includes:

- source chunks
- retrieval similarity scores
- prompt text
- model name
- request latency
- retrieval latency
- model latency
- prompt tokens
- completion tokens
- total tokens
- retrieval hit status
- quality score

These fields turn a model response into an observable platform event.

That is the AI reliability loop.

The system retrieves context, generates an answer, records the signals, and returns enough information to inspect whether the answer was grounded, timely, and reasonable.

This matters because AI reliability is not only uptime.

A system can be available and still produce weak answers.

It can return a response and still fail to retrieve useful context.

It can generate fluent text and still lack source support.

It can work functionally while using too many tokens.

It can answer correctly while becoming too slow under load.

The reliability loop gives the platform the signals needed to detect those conditions.

For an AI Data Platform Architect, this is one of the most important design patterns in the project. The platform should not treat the answer as the only output. It should treat the full request as a measurable event.

Async Work and Backpressure

The capstone includes a worker service to represent long-running AI jobs.

This is important because not every AI workload belongs in the synchronous request path.

Some work should happen in the background:

- document ingestion
- batch summarization
- embedding refreshes
- evaluation jobs
- workflow automation
- report generation
- long-running enrichment tasks

Separating this work from the API path protects user-facing latency.

The API can accept work, return status, and remain responsive while workers process tasks independently.

Redis provides the queue coordination point.

The worker service consumes work from the queue and interacts with persistent storage as needed.

This design introduces backpressure into the platform. If background work grows, workers can scale separately from the gateway. The platform does not need to overload the user-facing API with long-running execution.

That separation is one of the differences between a demo and a platform.

A demo often does everything inline.

A platform creates an execution path for slow work.

Postgres and Redis as Platform Dependencies

Postgres and Redis play different roles in the platform.

Postgres represents durable data and pgvector-ready storage.

It provides a foundation for document metadata, source context, application state, and other persistent records. In an AI platform, durable state matters because the system needs to preserve source references, operational events, job records, and retrieval-related data.

Redis represents queue coordination.

It supports the async worker path by giving the API and worker a lightweight coordination layer. The API can enqueue work, and the worker can process it independently.

This separation reflects a broader platform principle:

Durable state and queue coordination should not be treated as the same thing.

Postgres remembers.

Redis coordinates.

Together, they support a platform that can handle both user-facing requests and background work.

Observability as a Built-In Layer

The capstone includes a local observability dashboard.

This is not an optional add-on. It is part of the platform.

The observability layer gives visibility into how the system behaves across retrieval, generation, latency, token usage, and reliability fields.

The dashboard provides a way to inspect whether the platform is operating as expected.

Important signals include:

- request latency
- retrieval latency
- model latency

- token usage
- retrieval hit status
- quality score
- error behavior
- recent request events

This matters because AI systems fail in ways that are not always captured by traditional API metrics.

A request can return successfully while retrieval quality is poor.

A model can respond fluently while using too many tokens.

A platform can remain online while latency becomes unacceptable.

A dashboard makes those patterns visible.

From an AI Data Platform Architect's point of view, observability should be designed into the platform from the start. It should not arrive only after users complain about slow or low-quality answers.

Benchmarking the Platform

The capstone includes a benchmark script that targets the `/ask` endpoint.

The benchmark captures:

- average latency
- p95 latency
- requests per second
- error count
- token usage

This connects platform behavior to measurable performance.

The important architectural idea is not just running a load test.

The important idea is learning how to interpret where time is spent.

Retrieval latency points to document search and embedding behavior.

Model latency points to inference runtime and token generation.

API latency points to orchestration and metrics overhead.

Error rates point to dependency saturation, missing backpressure, or service failures.

Token counts point to cost, prompt size, and model-serving pressure.

Benchmarking helps the platform team move from subjective impressions to evidence.

Instead of saying “the platform feels slow,” the team can ask:

Is retrieval slow?

Is generation slow?

Is orchestration overhead high?

Are token counts increasing?

Are errors appearing under load?

Is p95 latency diverging from average latency?

That is how performance work becomes actionable.

Docker Compose for Local Integration

Docker Compose gives the project a way to run the full platform locally.

The local stack includes:

- API gateway
- worker
- LLM service
- RAG service
- observability service
- Postgres
- Redis

This is important because the capstone is not just testing one component.

It is testing service integration.

Docker Compose allows the platform to validate whether the gateway can reach retrieval, whether retrieval can use persistent storage, whether the gateway can call the LLM service, whether metrics are captured, whether Redis is available for worker jobs, and whether the dashboard can surface operational signals.

This local integration layer is valuable before moving to Kubernetes.

It allows the platform to debug service contracts, environment variables, dependency wiring, and basic runtime behavior in a simpler environment.

A local platform should be easy to start, inspect, and iterate on.

Docker Compose supports that workflow.

Kubernetes Deployment Readiness

The capstone also includes Kubernetes manifests that translate the local stack into a production-style deployment shape.

The Kubernetes files define:

- namespace isolation
- ConfigMaps
- Secrets
- Deployments
- Services
- Ingress
- HorizontalPodAutoscalers
- NetworkPolicies

This creates a path from local development to production hardening.

The Kubernetes manifests are not the final production state. They establish the operational boundaries.

They show how each component can be deployed independently.

They show how routing can be defined.

They show how configuration and secrets can be externalized.

They show how services can be scaled.

They show how network communication can begin from a more controlled baseline.

This is the right progression for an AI infrastructure learning lab.

First, build the capabilities locally.

Then integrate them.

Then define deployment boundaries.

Then document the gaps before production.

That is how a platform matures.

Why This Matters for AI Data Platforms

AI data platforms are not just about calling models.

They are about creating reliable systems around models.

A platform needs to support:

- clear service boundaries
- source-grounded retrieval
- model generation
- token accounting
- async processing
- queue coordination
- durable state
- latency visibility
- quality signals
- benchmarking discipline
- local integration
- deployment readiness

The capstone demonstrates how these pieces fit together.

It shows that a model endpoint is only one part of the system.

The platform also needs routing, retrieval, observability, background processing, performance measurement, and deployment controls.

That is the difference between an AI feature and AI infrastructure.

An AI feature can generate a response.

AI infrastructure can generate a response, cite the sources, measure the cost, track the latency, expose reliability fields, support background work, and prepare the system for deployment.

Architect's Takeaway

The Observable Local LLM Platform demonstrates the practical mindset needed for AI infrastructure work:

Build the model path, but also build the operating layer around it.

A reliable AI platform needs:

- clear service boundaries
- source-grounded retrieval
- measured model behavior
- async processing
- token and latency visibility
- benchmarking discipline
- Dockerized local development
- Kubernetes deployment readiness
- honest production-readiness gaps

The capstone brings those ideas together into one local platform.

It shows how AI applications become platforms.

The final lesson is straightforward:

Models power the user experience, but infrastructure makes that experience dependable.

A production-minded AI system needs more than a working prompt and a model endpoint. It needs a platform that can route requests, retrieve sources, generate answers, measure behavior, process long-running work, expose operational signals, and define a path toward deployment.

That is what this capstone is designed to demonstrate.